Ex3b: Model Planning and Building – Regression

Dataset: Advertising Dataset (Kaggle) – `Advertising.csv`

Upload `Advertising.csv` to the Colab **Files** panel before running. If the file is not uploaded, the notebook downloads the same dataset automatically.

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.svm import SVR
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import warnings
warnings.filterwarnings('ignore')

sns.set_style('whitegrid')
```

Task 1: Load the Advertising dataset into a Pandas DataFrame

```
file_name = "Advertising.csv"
if not os.path.exists(file_name):  # file not uploaded -> read the same dataset from GitHub
    file_name = "https://raw.githubusercontent.com/justmarkham/scikit-learn-videos/master/data/Advertising.csv"

df = pd.read_csv(file_name)
df = df.loc[:, ~df.columns.str.contains('^Unnamed')]   # drop the unnamed index column if present
df.columns = [c.strip().capitalize() if c.strip().upper() != 'TV' else 'TV' for c in df.columns]
print("Dataset loaded. Columns:", df.columns.tolist())
```

```
Dataset loaded. Columns: ['TV', 'Radio', 'Newspaper', 'Sales']
```

Task 2: Explore the dataset

First and last five records

```
print("First five records:")
display(df.head())
print("Last five records:")
display(df.tail())
```

```
First five records:
```

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5
4	180.8	10.8	58.4	12.9

```
Last five records:
```

	TV	Radio	Newspaper	Sales
195	38.2	3.7	13.8	7.6
196	94.2	4.9	8.1	9.7
197	177.0	9.3	6.4	12.8
198	283.6	42.0	66.2	25.5
199	232.1	8.6	8.7	13.4

Shape and data types

```
print("Shape of the dataset:", df.shape)
print("\nData types:")
print(df.dtypes)
```

```
Shape of the dataset: (200, 4)

Data types:
TV           float64
Radio        float64
Newspaper    float64
```

```
Sales        float64
dtype: object
```

Missing values and duplicate records

```
print("Missing values in each column:")
print(df.isnull().sum())
print("\nNumber of duplicate records:", df.duplicated().sum())
```

```
Missing values in each column:
TV           0
Radio        0
Newspaper    0
Sales        0
dtype: int64

Number of duplicate records: 0
```

Summary statistics

```
display(df.describe().T)
```

	count	mean	std	min	25%	50%	75%	max
TV	200.0	147.0425	85.854236	0.7	74.375	149.75	218.825	296.4
Radio	200.0	23.2640	14.846809	0.0	9.975	22.90	36.525	49.6
Newspaper	200.0	30.5540	21.778621	0.3	12.750	25.75	45.100	114.0
Sales	200.0	14.0225	5.217457	1.6	10.375	12.90	17.400	27.0

Task 3: Select TV, Radio, Newspaper as input features (X) and Sales as target (y)

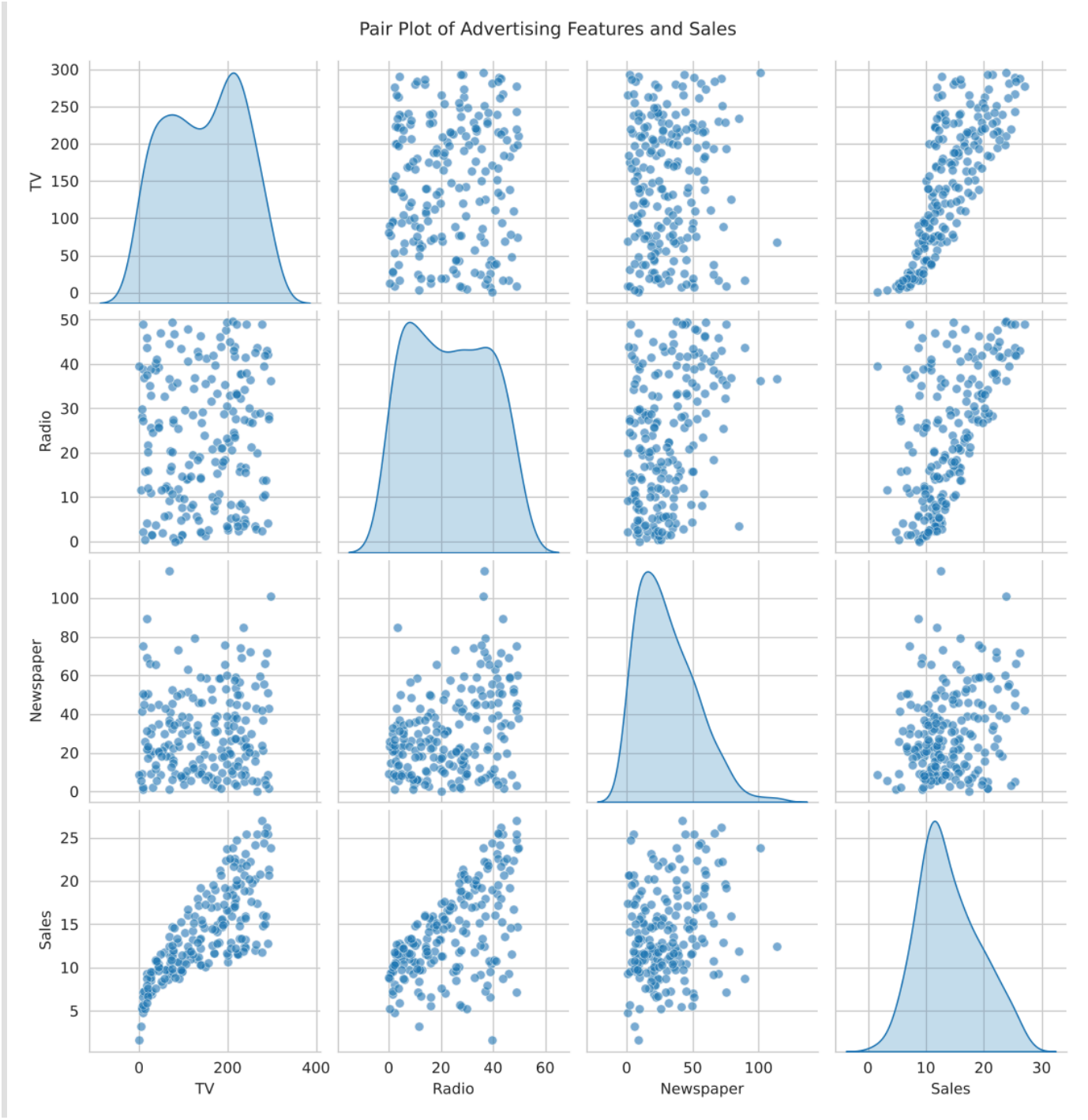
```
X = df[['TV', 'Radio', 'Newspaper']]
y = df['Sales']
print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (200, 3)
y shape: (200,)
```

Exploratory Data Analysis

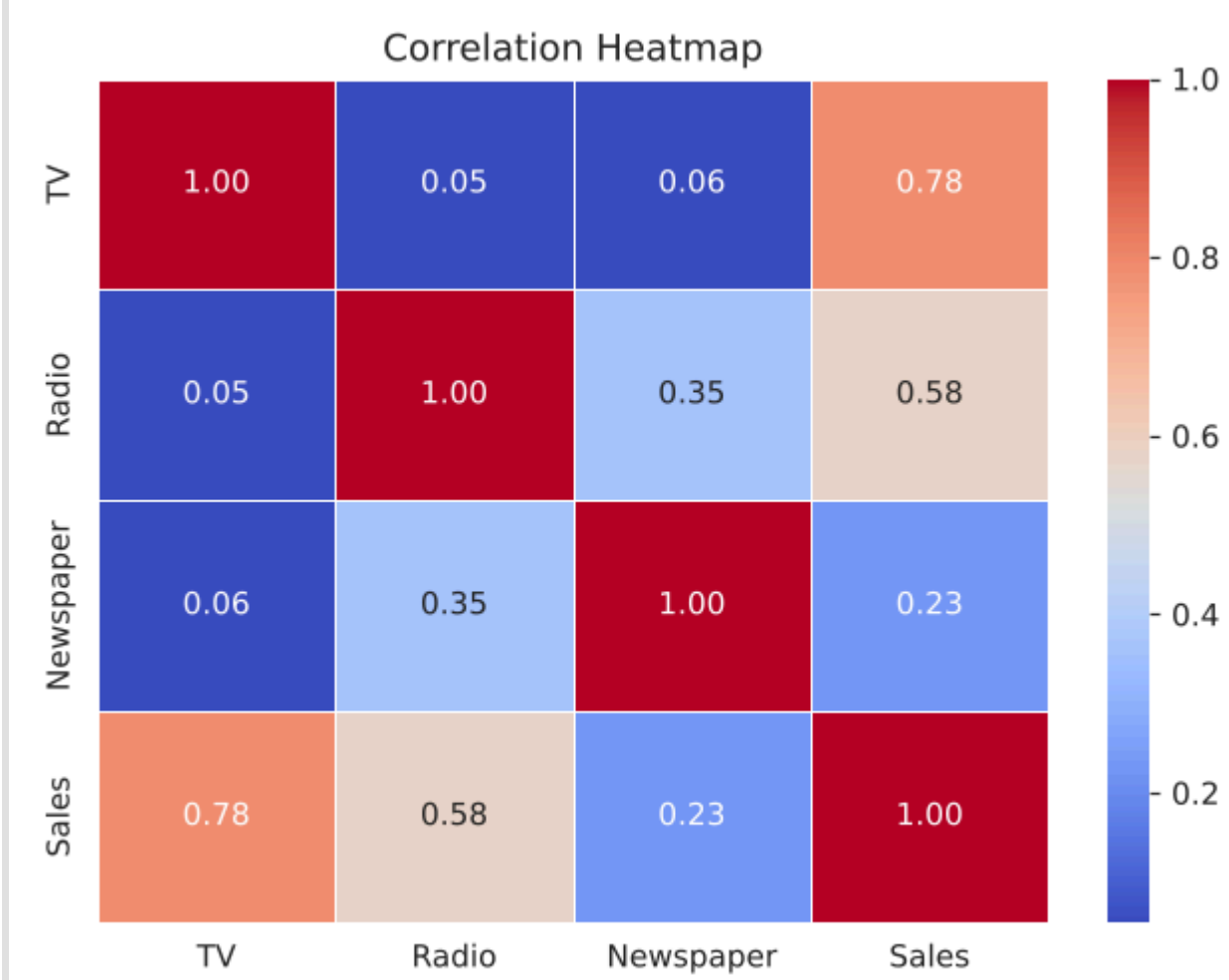
Task 4: Pair Plot of the selected features

```
sns.pairplot(df[['TV', 'Radio', 'Newspaper', 'Sales']], diag_kind='kde', plot_kws={'alpha': 0.6})
plt.suptitle('Pair Plot of Advertising Features and Sales', y=1.02)
plt.show()
```



Task 5: Correlation Heatmap

```
corr = df[['TV', 'Radio', 'Newspaper', 'Sales']].corr()
plt.figure(figsize=(7, 5))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Heatmap')
plt.show()
```



Task 6: Feature most strongly correlated with Sales

```
sales_corr = corr['Sales'].drop('Sales').sort_values(ascending=False)
print("Correlation of each feature with Sales:")
print(sales_corr.round(4))
print(f"\nFeature most strongly correlated with Sales: {sales_corr.idxmax()} (r = {sales_corr.max():.4f})")
```

```
Correlation of each feature with Sales:
TV           0.7822
Radio        0.5762
Newspaper    0.2283
Name: Sales, dtype: float64

Feature most strongly correlated with Sales: TV (r = 0.7822)
```

Model Planning and Model Building

Task 7: Split the dataset into Training (80%) and Testing (20%)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("X_train:", X_train.shape, " X_test:", X_test.shape)
print("y_train:", y_train.shape, " y_test:", y_test.shape)
```

```
X_train: (160, 3)  X_test: (40, 3)
y_train: (160,)  y_test: (40,)
```

Task 8: Linear Regression

```
lin_model = LinearRegression()
lin_model.fit(X_train, y_train)
y_pred_lin = lin_model.predict(X_test)

print("Intercept:", round(lin_model.intercept_, 4))
print("Coefficients:", {col: round(float(c), 4) for col, c in zip(X.columns, lin_model.coef_)})
display(pd.DataFrame({'Actual Sales': y_test.values, 'Predicted Sales': y_pred_lin.round(2)}).head(10))
```

```
Intercept: 2.9791
Coefficients: {'TV': 0.0447, 'Radio': 0.1892, 'Newspaper': 0.0028}
```

	Actual Sales	Predicted Sales
0	16.9	16.41
1	22.4	20.89
2	21.4	21.55
3	7.3	10.61
4	24.7	22.11
5	12.6	13.11
6	22.3	21.06
7	8.4	7.46
8	11.5	13.61
9	14.9	15.16

Task 9: Polynomial Regression (Degree = 2)

```
poly_model = make_pipeline(PolynomialFeatures(degree=2), LinearRegression())
poly_model.fit(X_train, y_train)
y_pred_poly = poly_model.predict(X_test)

display(pd.DataFrame({'Actual Sales': y_test.values, 'Predicted Sales': y_pred_poly.round(2)}).head(10))
```

	Actual Sales	Predicted Sales
0	16.9	17.25
1	22.4	22.72
2	21.4	20.43
3	7.3	7.54
4	24.7	24.37
5	12.6	12.56
6	22.3	22.81
7	8.4	8.34
8	11.5	12.06
9	14.9	15.69

Task 10: Support Vector Regression (RBF kernel)

SVR is distance-based, so the features are standardised with `StandardScaler` inside the pipeline.

```
svr_model = make_pipeline(StandardScaler(), SVR(kernel='rbf'))
svr_model.fit(X_train, y_train)
y_pred_svr = svr_model.predict(X_test)

display(pd.DataFrame({'Actual Sales': y_test.values, 'Predicted Sales': y_pred_svr.round(2)}).head(10))
```

	Actual Sales	Predicted Sales
0	16.9	17.19
1	22.4	21.60
2	21.4	19.58
3	7.3	9.77
4	24.7	19.95
5	12.6	12.50
6	22.3	21.91
7	8.4	8.54
8	11.5	12.05
9	14.9	15.49

Model Evaluation

Task 11: MAE, MSE, RMSE and R² Score for all three models

```
predictions = {
    'Linear Regression': y_pred_lin,
    'Polynomial Regression (deg 2)': y_pred_poly,
    'SVR (RBF)': y_pred_svr,
}

rows = []
for name, y_pred in predictions.items():
    mae = mean_absolute_error(y_test, y_pred)
```

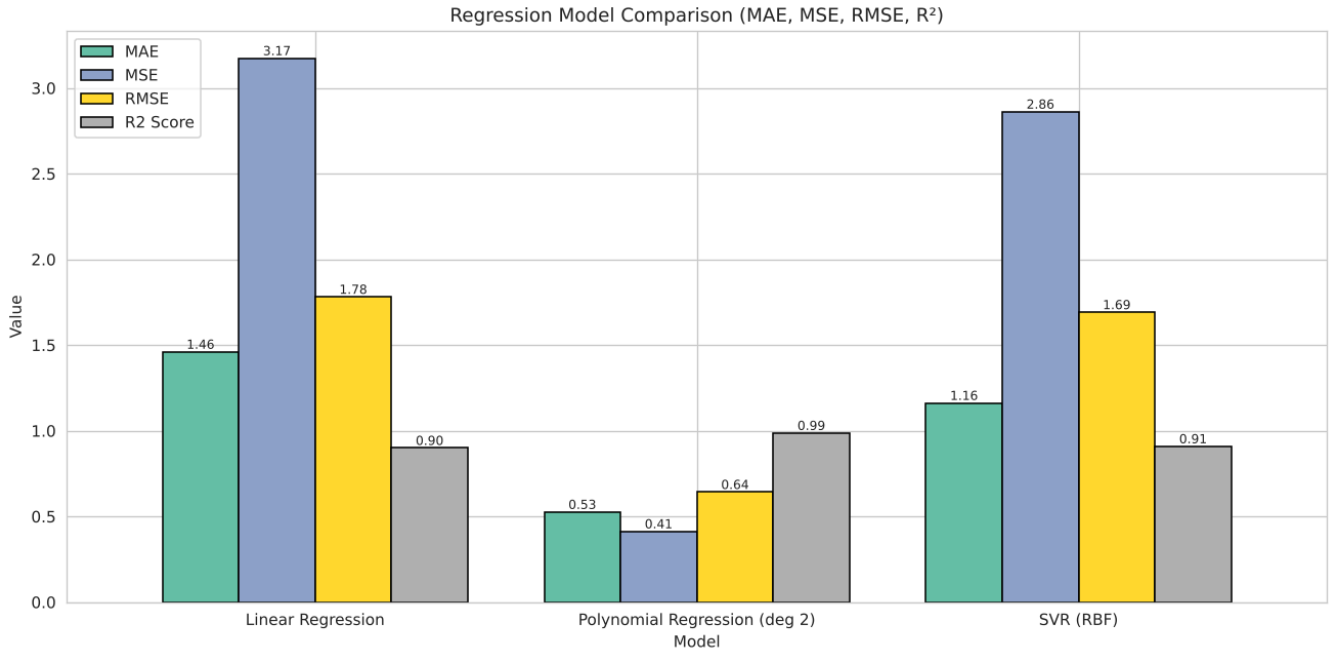
```
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)
    rows.append([name, mae, mse, rmse, r2])

metrics_df = pd.DataFrame(rows, columns=['Model', 'MAE', 'MSE', 'RMSE', 'R2 Score']).set_index('Model')
display(metrics_df.round(4))
```

Model	MAE	MSE	RMSE	R2 Score
Linear Regression	1.4608	3.1741	1.7816	0.8994
Polynomial Regression (deg 2)	0.5262	0.4129	0.6426	0.9869
SVR (RBF)	1.1621	2.8621	1.6918	0.9093

Task 12: Compare all models using a single grouped bar chart

```
ax = metrics_df.plot(kind='bar', figsize=(12, 6), colormap='Set2', edgecolor='black', width=0.8)
for container in ax.containers:
    ax.bar_label(container, fmt='%.2f', fontsize=8)
plt.title('Regression Model Comparison (MAE, MSE, RMSE, R²)')
plt.ylabel('Value')
plt.xlabel('Model')
plt.xticks(rotation=0)
plt.legend(loc='upper left')
plt.tight_layout()
plt.show()
```



Task 13: Actual vs Predicted Sales for the best-performing model

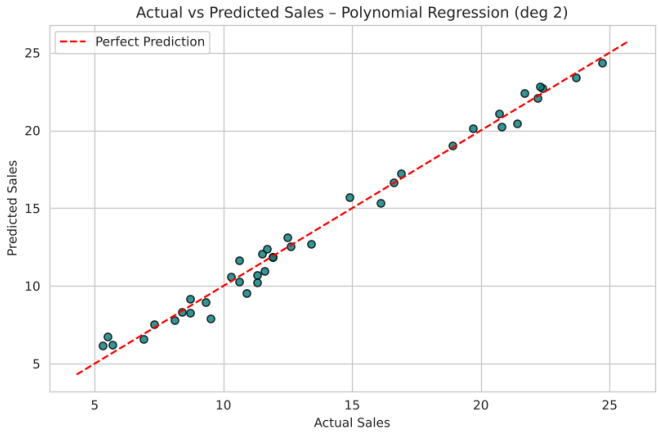
```
best_model_name = metrics_df['RMSE'].idxmin()
best_pred = predictions[best_model_name]

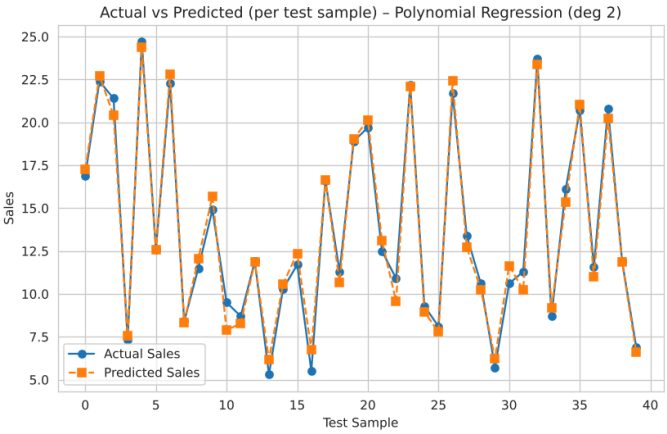
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].scatter(y_test, best_pred, color='teal', edgecolor='black', alpha=0.8)
lims = [min(y_test.min(), best_pred.min()) - 1, max(y_test.max(), best_pred.max()) + 1]
axes[0].plot(lims, lims, 'r--', label='Perfect Prediction')
axes[0].set_xlabel('Actual Sales')
axes[0].set_ylabel('Predicted Sales')
axes[0].set_title(f'Actual vs Predicted Sales — {best_model_name}')
axes[0].legend()

axes[1].plot(range(len(y_test)), y_test.values, 'o-', label='Actual Sales')
axes[1].plot(range(len(y_test)), best_pred, 's--', label='Predicted Sales')
axes[1].set_xlabel('Test Sample')
axes[1].set_ylabel('Sales')
axes[1].set_title(f'Actual vs Predicted (per test sample) — {best_model_name}')
axes[1].legend()

plt.tight_layout()
plt.show()
```





Model Interpretation

Task 14: Model with the lowest prediction error

```
print(f"Model with the lowest prediction error: {best_model_name}")
print(f"  MAE  = {metrics_df.loc[best_model_name, 'MAE']:.4f}")
print(f"  MSE  = {metrics_df.loc[best_model_name, 'MSE']:.4f}")
print(f"  RMSE = {metrics_df.loc[best_model_name, 'RMSE']:.4f}")
print(f"  R²   = {metrics_df.loc[best_model_name, 'R2 Score']:.4f}")
```

```
Model with the lowest prediction error: Polynomial Regression (deg 2)
  MAE  = 0.5262
  MSE  = 0.4129
  RMSE = 0.6426
  R²   = 0.9869
```

Task 15: Recommendations to optimise the advertising budget

1. **Invest mainly in TV and Radio.** TV has the strongest correlation with Sales, and Radio has the largest per-unit coefficient in the linear model, so these two channels give the best return on spend.
2. **Cut or reallocate the Newspaper budget.** Newspaper has a weak correlation with Sales and a coefficient close to zero, so money spent there adds little to sales and should be moved to TV / Radio.
3. **Run TV and Radio campaigns together and plan with the polynomial model.** The degree-2 polynomial model (which captures the TV × Radio interaction) has the lowest error, which shows that the two channels boost each other. Combine them in the same campaigns and use this model to test different budget splits and predict sales before spending.